SE301 ADVANCED PROGRAMMING AND DESIGN · PROJECT 2026-27

Singapore Vertical Farming Research Management System

Legacy Refactoring & Modernisation Report

Group	G1 Team 3
Member 1	Etienne Wong Ai Ting
Member 2	Naren Swaminathan
Member 3	Anson Poh Jing En

1. Legacy System Analysis

SVFRMS simulates one day of an indoor vertical farm. Four chamber threads (CH1 Tomato, CH2 Strawberry, CH3 Lettuce, CH4 Chili) each run 20 cycles. Each cycle reads five sensors (temperature, humidity, soil moisture, light, nutrient), checks every reading against hard-coded crop thresholds, raises alerts, charges water and energy, and puts device requests (for example `"CH1|Cooling Fan"`) on a bounded queue of 5. A control thread takes requests off the queue and runs the device, which changes the chamber's state. At the end, two racing threads archive the alerts and print the daily report, and `archive/daily_report.txt` is written.

Everything is in one 393-line class, `SVFRMS.java`. Its main components are global `public static` state (lines 7–31), the `ChamberState` / `ChamberSummary` model (33–46), a string-switched event hub `handleEvent` (62–88), the rule engine `processSensor` (141–205), device control `activateControl` (90–129), a hand-written queue and `ControlThread` (131–139, 251–273), `ChamberThread` (275–295), reporting (207–241, 309–347), dead code (243–249, 297–307), and `main` (349–392).

2. Code Analysis and Limitations

2.1 Design problems (SOLID, spec 4.2)

Problem	Evidence in legacy code	Principle
God class	One class holds the model, rules, devices, queue, threads, reports and file I/O. Any change means retesting everything.	SRP (i)
Crop `if/else` chains	Adding a crop means editing `processSensor` in up to 3 places (147–187). The same alert block is copy-pasted 19 times, with thresholds and costs as magic numbers.	OCP (ii), DRY
Device `if/else` chain	`activateControl` (90–129) must be edited for every new device; each branch repeats the same three `handleEvent` calls.	OCP (ii), SRP
`extends Thread`	`ControlThread` swallows `interrupt()` and loops `while(true)`, so it does not honour the `Thread` cancellation contract (251, 258).	LSP (iii)
One wide entry point	`handleEvent(type, chamber, crop, msg)` does logging, storage, counting and printing chosen by a string; callers pass dummy `""` (95, 142, 148).	ISP (iv), SRP
Global mutable state	Static lists, maps, counters, `System.out`, `new Date()`, a static `Random`: nothing can be tested or replaced.	DIP (v)
Stringly-typed messages	Queue items are `chamber + "|" + device`, split again by the consumer (136, 269).	–
Duplicated report, dead code	Report format written twice (210–214 vs 314–319); four unused methods and two unused locks.	SRP

2.2 Concurrency problems (spec 4.4)

We ran the unmodified program 300 times and checked each run against invariants derived from its own archived alert list. **Only 92 of 300 runs were correct.**

#	Issue	Effect	Observed
C1	Check-then-act on `HashMap` (`containsKey → put → get`, 48–60) from 4 threads	`get` returns `null`; the chamber thread dies with an NPE (146, 80) and the chamber vanishes from the report	123/300 runs
C2	Unsynchronized `ArrayList.add` (64, 73)	Lost alerts: generated count ≠ archived count	67/300
C3	Non-atomic `total += n` counters	Lost water/energy updates	27 / 24 of 300
C4	`ChamberState` written by the chamber and control threads without a lock	`temperature - 5` uses a stale value or overwrites a newer reading	yes
C6	Archive and report threads started together (384–386); a lock gives exclusion, not order	Report sometimes printed before archiving	7/300
C7	`alertLock` taken by readers but never by writers	False sense of safety	inspection

C9–C10	`InterruptedException` swallowed in both `wait()` loops; if the control thread dies, producers block forever on the full queue	Uncancellable threads; program hang	reachable
C11	No failure handling for threads	A crashed chamber still produces a "successful" report	every C1 run

Deadlock assessment. The only nested locking is `alertLock → reportLock`, taken in the *same* order in `generateDailyReport` and `archiveAlerts`, so no cycle can form and no deadlock is reachable as written. It is one edit away from the classic deadlock: if either method took `reportLock` first, each report thread could hold one lock and wait for the other. The real liveness risk is C10. Our design removes lock nesting entirely (section 5).

Spec gaps. The code has no severity levels and no sensor-failure alert (spec 3.4); we add both as an extension that leaves the legacy output untouched (section 6). Other gaps are documented, not changed, because spec 4 requires preserved behaviour: Heating Unit, Ventilation and Shade Controller do not exist (Humidifier and Dehumidifier do), only Tomato has every rule, and there is one combined report.

3. Proposed Design

Design rule: preserve behaviour exactly, and fix every smell and concurrency defect above. Every class exists to remove a specific problem. We captured the legacy output with a fixed seed and serial schedule, and our refactored output is **byte-identical after normalization** (timestamps and control-thread timing removed).

- **Crop rules as data.** All 19 copied alert blocks became rows in `crops.csv`: `crop, sensor, HIGH|LOW, limit, device, water, energy, message`. `*` rows apply to every crop (the legacy global light and nutrient limits). `SensorProcessor` loops over whatever rules the registry holds and never names a crop. We chose a CSV over a Java catalogue class because adding a crop to a class is still "modifying existing code" (spec 4.2 ii).
- **One class per device** behind a two-method `Device` interface. `ControlService` prints and records every activation in one place.
- **Observer for alerts.** The rule engine publishes one `Alert`; the summary tracker, the alert store and the console printer each do one job.
- **Objects, not strings, on the queue.** A `ControlRequest` holds the chamber, its state and the device.
- **Composition root.** Only `SVFRMS.createSimulation` creates and connects objects; everything else gets its dependencies through its constructor, so the tests and the baseline harness can inject a seeded or fixed sensor source.
- **Fail fast on bad configuration.** Every CSV error (unknown device or sensor, wrong column count, duplicate chamber) stops the program before any thread starts, naming the file and line; a chamber whose crop has no rows gets a warning.

3.1 SOLID mapping

Spec	Change
(i) SRP	About 30 small classes in 7 packages, each with one job. All report formatting lives in `DailyReport` only.
(ii) OCP	New crop = rows in `crops.csv`. New device = new `Device` class + one registration. New alert output = new `AlertListener`. No existing class changes.
(iii) LSP	No `Thread` subclasses: tasks are `Runnable`s run by an `ExecutorService`, and the consumer exits on interrupt. `Device` has only methods every device can honour (no `increase()/decrease()` pair a fan cannot implement).
(iv) ISP	`handleEvent` is replaced by three tiny interfaces: `SensorSource`, `Device`, `AlertListener`.
(v) DIP	`SensorProcessor` depends on `SensorSource`, `Device` and `AlertListener` abstractions; the output and error `PrintStream`s, the `Clock` used for timestamps and the sensor source are all injected.

4. Final Architecture

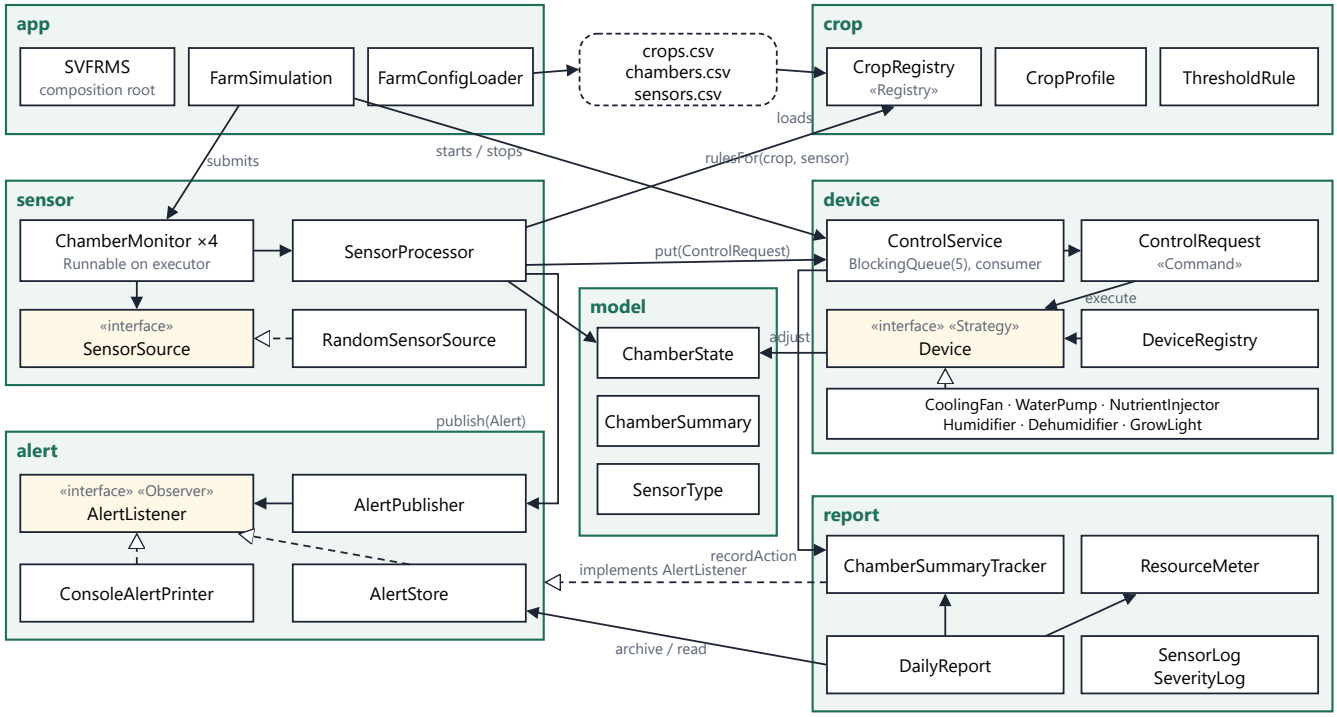


Figure 1. Component diagram. Yellow boxes are interfaces; dashed arrows with hollow heads mean "implements".

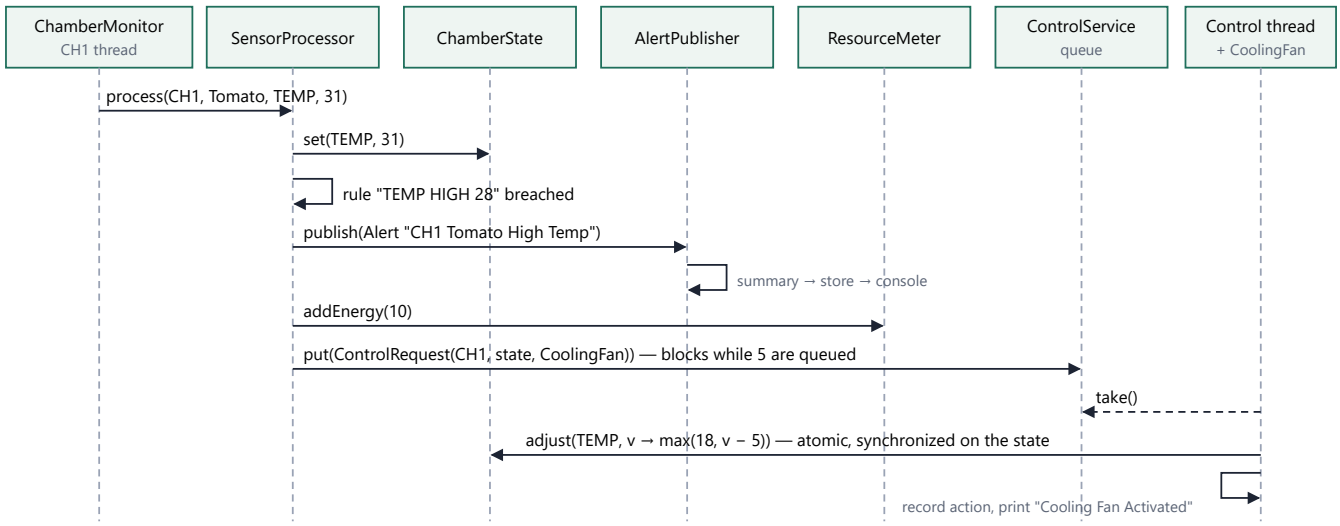


Figure 2. Sequence: one high-temperature reading in CH1. No lock is held while publishing or queueing.

End of the run (`FarmSimulation.run`): start the control consumer; submit one `ChamberMonitor` per row of `chambers.csv` and wait on each `Future` ; call `control.stop()` , which queues a stop marker *behind* all pending requests so the queue drains first (giving up after 30 s if a device hangs), and check the control thread's own `Future` ; then, on the main thread and in this fixed order, archive the alerts, print the report and save the file, plus `alerts_by_severity.txt` . If any chamber or the control thread failed, the error is printed and the program exits with status 1.

5. Concurrency Fixes

Issue	Fix	Tool
C1	Summaries created with atomic `computeIfAbsent` . Chamber state is never looked up: each monitor owns its `ChamberState` and passes it inside the `ControlRequest` .	`ConcurrentHashMap`
C2, C7	All alert-store methods synchronized on one lock that guards active list, archive and count together; the sensor log is a synchronized list.	intrinsic lock
C3	Water and energy totals.	`AtomicInteger.addAndGet`

C4, C5	`ChamberState.set/adjust` and `ChamberSummary` methods are synchronized; a device's read-modify-write is one call to `adjust`.	one lock per chamber
C6	Archive, report and save run sequentially on the main thread after all workers finish.	program order
C8–C10	Unused locks removed. Hand-written buffer replaced. The consumer catches a failing request and keeps going; executors shut down with a timeout. Interrupts are never swallowed: a producer interrupted on a full queue keeps the flag and fails loudly (no silently lost request), if the consumer dies, every request still queued is reported and the run fails; and if the stop marker cannot be queued within 30 s the consumer is interrupted instead. An interrupted run stops the chambers before the consumer and still writes its report.	`ArrayBlockingQueue`, `ExecutorService`
C11	`Future.get()` re-throws a chamber's exception; it is reported and the exit status is non-zero.	`Future`

Deadlock freedom by construction. No method holds two locks at once. Each lock (`ChamberState`, `ChamberSummary`, `AlertStore`, the sensor log) is held only for a few field updates and never while calling another object; `SensorProcessor` holds no lock while it publishes an alert or waits to queue a request. With at most one lock held at a time, the lock-wait graph cannot contain a cycle.

> **Evidence.** Legacy: 92/300 concurrent runs correct. Refactored `run.jar`: **300/300** correct with the 4 legacy chambers and **300/300** with the 7 demo chambers, with the same invariant checker. Seeded serial run on the 4 legacy chambers: normalized output **identical** to the legacy baseline. 48 JUnit test methods (60 runs, as the stress tests repeat 3 times), including stress tests that hammer every shared object from 8 threads × 5,000 operations, a producer-consumer test with a queue of 1, a failing-device test and a crashed-consumer test (C10), an interrupted-producer test (C9), a failing-chamber test (C11) and an interrupted-run test.

6. Extensibility and Maintainability

Change	Legacy	Refactored
Add a crop (Kale)	Edit `processSensor` in up to 3 places	Add rows to `crops.csv`, and a row such as `CH5, Kale` to a `chambers.csv`
Remove a crop	Delete branches and hope nothing else uses them	Delete its rows
Change a threshold or cost	Find the right `if` among 19 copies	Edit one number in one row
Add a device (Heating Unit)	Edit `activateControl`	New `HeatingUnit implements Device`, register it in `SVFRMS`, use its name in a CSV row
Add an alert output (file, SMS)	Edit `handleEvent`	New `AlertListener`, subscribe it in `SVFRMS` (done for severity, below)
Use real sensors	Rewrite `ChamberThread`	Implement `SensorSource`, pass it to `createSimulation`
Change the report format	Edit two places	Edit `DailyReport`

Crop extension (spec 4.5). Kale, Bok Choy, Bell Pepper and Eggplant were added as rows in `crops.csv`, with **no Java changes**; each automatically gets the shared light and nutrient rules. The default `chambers.csv` keeps the 4 legacy chambers, so a plain run behaves exactly as before. `java -jar run.jar demo` reads `demo/chambers.csv`, which adds `CH5, Kale`, `CH6, Bok Choy` and `CH7, Bell Pepper` without a rebuild (a notice names the override file); this 7-chamber layout also passes 300/300 concurrent runs. Only an explicitly named folder is read, so a stray CSV can never change a normal run. A chamber whose crop has no rows (a removed crop or a typo) keeps only the shared rules, as in the legacy code, and a start-up warning names the line.

```
Kale, SOIL, LOW, 40, Water Pump, 30, 0, WARNING, {crop} Low Soil Moisture  →  [CH5][Kale][ALERT] CH5 Kale Low Soil Moisture
```

Alerting extension (spec 3.4). Each rule now carries a severity column (temperature CRITICAL, humidity, soil and nutrient WARNING, light INFO). A new `SeverityLog` listener writes every alert with its level and a count per level to `archive/alerts_by_severity.txt`. A sensor that does not respond, or reports a reading outside its valid range in `sensors.csv` (NaN, humidity outside 0–100 %), raises a CRITICAL *Sensor Failure* alert; nothing is stored, no device runs and the chamber carries on. One extra console line ends the run, e.g. `Alerts by severity: INFO=21, WARNING=64, CRITICAL=93`; every legacy console line and the daily report are still identical to the legacy baseline.

7. Design Patterns

Pattern	Where	Problem it solves and why we chose it
Strategy	`Device` + 6 classes	The `activateControl` chain. Each device is a different algorithm behind one interface, so a new device never touches `ControlService`.
Observer	`AlertPublisher` → `AlertListener`s	`handleEvent` did four jobs behind a string switch. The rule engine now raises one alert, and each listener (summary, store, console) does one job. New outputs are new listeners.
Command	`ControlRequest`	Replaces the `"CH1|Cooling Fan"` string. The request carries the device and the chamber state and has `execute()`, so the producer decides *what* and the consumer decides *when*.
Producer–Consumer	`ControlService`	Replaces the hand-written `wait/notifyAll` buffer that swallowed interrupts, with the JDK's `ArrayBlockingQueue` (capacity 5, as before).
Registry	`CropRegistry`, `DeviceRegistry`	Crop and device names were hard-coded in control flow. A lookup by name is all that is needed; no factory hierarchy.
Composition root (DI)	`SVFRMS.createSimulation`	Replaces global static state. One place builds the object graph, so tests and the baseline harness can swap the sensor source.

Not used on purpose: a Strategy class per crop (crops differ only in numbers, so data is simpler); Singleton (it would recreate the global state we removed); Factory (nothing needs a family of products).

Build and run: `mvn package` (runs the tests and builds `target/run.jar`), then `java -jar run.jar` (or `java -jar run.jar demo` for the new crops). The baseline scripts and evidence are in `baseline/`; the full analysis is in `docs/phase1-analysis.md` and `docs/phase2-design.md`.